



Digitalna platforma za upravljanje teretanom

Dizajn sistema

Amelija Trifunović, Bogdana Marić, Milica Bobar, Jelena Matijaš, Jana Ivanković

Verzija: 1.0 | Datum: MAJ 2026.

Sadržaj

1 Uvod3

- 1.1 Namjena sistema3
- 1.2 Projektni ciljevi3
- 1.3 Definicije i skraćenice3
- 1.4 Referentni dokumenti4
- 1.5 Kratak pregled dokumenta i predložene arhitekture4

2 Arhitektura sistema5

- 2.1 Kratak pregled arhitekture i funkcionalnosti podsistema5
 - 2.1.1 Tehnologije i razlog odabira6
 - 2.1.2 Klijentski podsistem (mobilna aplikacija)6
 - 2.1.3 Backend podsistem7
- 2.2 Dekompozicija sistema8
- 2.3 HW/SW mapiranje9
 - 2.3.1 Dijagram komponenata9
 - 2.3.2 Dijagram razmještaja10
- 2.4 Perzistentni sloj11
- 2.5 Kontrola prava pristupa12
- 2.6 Identifikacija konkurentnosti14
- 2.7 Kontrola toka14
- 2.8 Granična stanja sistema15
 - 2.8.1 Inicijalizacija15
 - 2.8.2 Terminiranje15
 - 2.8.3 Otkazi15

1 Uvod

Svrha ovog dokumenta je da detaljno opiše arhitekturu i dizajn sistema pumpd — digitalne platforme za upravljanje teretanom — i da dopuni specifikaciju softverskih zahtjeva (SRS).

1.1 Namjena sistema

pumpd je multiplatformska mobilna aplikacija namijenjena digitalizaciji procesa upravljanja teretanom. Sistem pruža centralizovan pristup svim ključnim funkcionalnostima vezanim za članske usluge, praćenje treninga i upravljanje osobljem.

Ključne funkcionalnosti sistema su:

- Upravljanje članstvima: Evidencija i praćenje statusa članarina, rezervacija i otkazivanje termina.
- Praćenje treninga: Kreiranje, pregled i uređivanje planova treninga, uvid u prethodne treninge i napredak.
- Upravljanje trenerima: Pregled profila trenera, slanje zahtjeva za praćenje, ostavljanje recenzija.
- Grupni treninzi: Pregled dostupnih grupa, prijava i odjava sa termina, upravljanje grupama.
- Objave i novosti: Dijeljenje i čuvanje objava, personalizovana stranica vijesti.
- Administracija: Upravljanje nalogima članova, trenera i radnika, obrada prijava problema.

1.2 Projektni ciljevi

- Sigurnost: Zaštita korisničkih podataka kroz autentifikaciju (JWT tokeni), autorizaciju zasnovanu na ulogama (RBAC), kriptografsku zaštitu lozinki i validaciju ulaznih podataka.
- Lakoća upotrebe: Intuitivan i moderan mobilni interfejs prilagođen korisnicima bez tehničkog predznanja.
- Prenosivost: Dostupnost na Android i iOS platformama kroz jedinstven kôd (cross-platform razvoj).
- Prilagodljivost: Personalizovan prikaz sadržaja u zavisnosti od uloge korisnika.
- Proširivost: Modularna arhitektura koja omogućava jednostavno dodavanje novih funkcionalnosti.
- Lako održavanje: Primjena dobrih programerskih praksi, jasna separacija odgovornosti između slojeva.
- Dobra dokumentacija: Sveobuhvatna tehnička dokumentacija koja prati cijeli projektni ciklus.

1.3 Definicije i skraćenice

Skraćenica / Pojam	Objašnjenje
API	Application Programming Interface — skup pravila za komunikaciju između softverskih komponenti
REST	Representational State Transfer — arhitekturni stil za dizajn mrežnih servisa
HTTPS	Hypertext Transfer Protocol Secure — sigurna verzija HTTP protokola
JWT	JSON Web Token — standard za sigurno prenošenje informacija između strana kao JSON objekat
MVVM	Model-View-ViewModel — arhitekturni obrazac za mobilne aplikacije
ORM	Object-Relational Mapping — tehnika mapiranja objekata na tabele baze podataka
CRUD	Create, Read, Update, Delete — osnovne operacije nad podacima
RBAC	Role-Based Access Control — kontrola pristupa zasnovana na ulogama
GUI	Graphical User Interface — grafički korisnički interfejs
SQL	Structured Query Language — upitni jezik za rad s relacionim bazama podataka
APK	Android Package — format instalacionog paketa za Android platformu

1.4 Referentni dokumenti

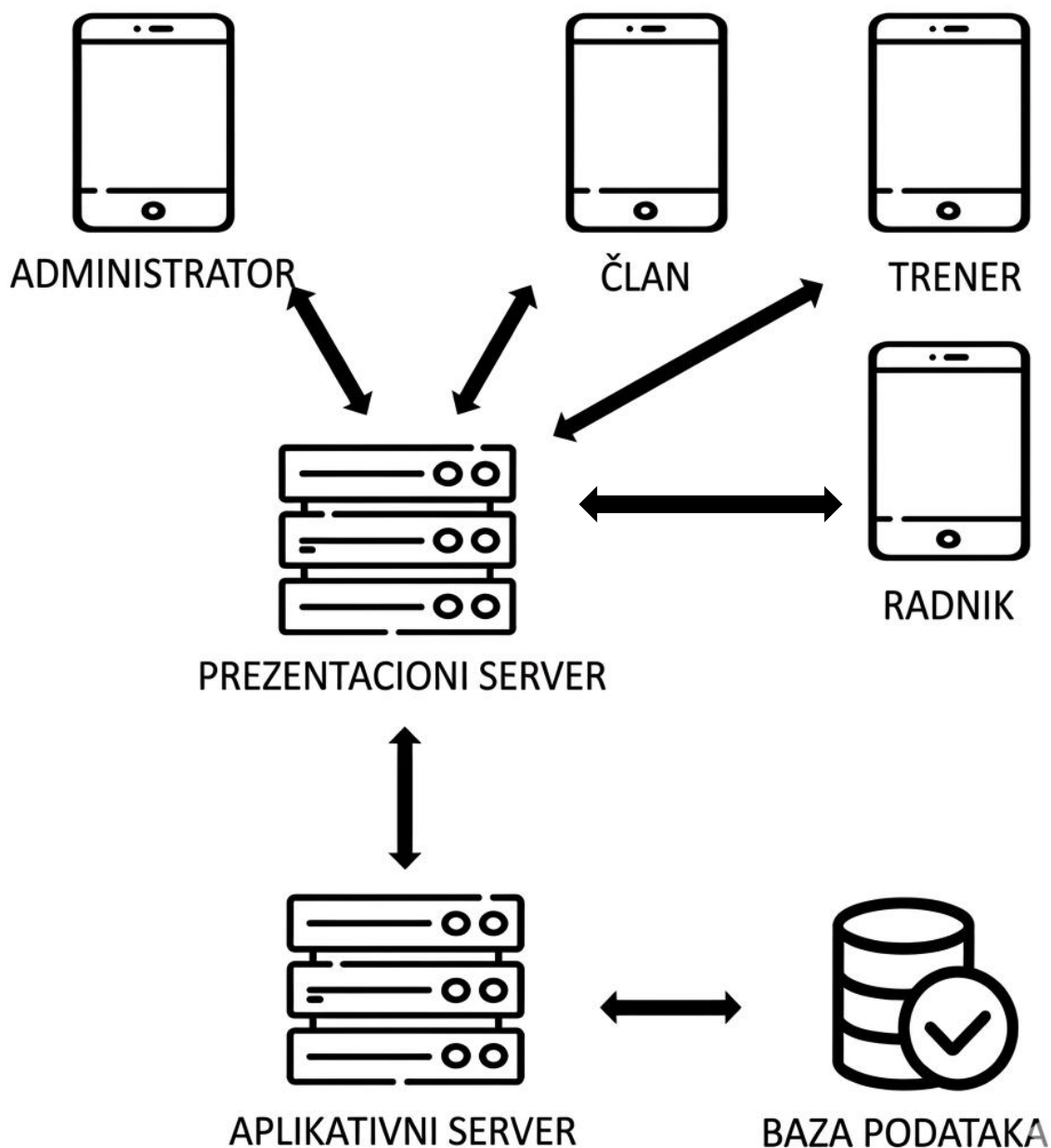
- Specifikacija softverskih zahtjeva — pumpd, Digitalna platforma za upravljanje teretnom
- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- <https://reactnative.dev/docs/getting-started>

1.5 Kratak pregled dokumenta i predložene arhitekture

Ovaj dokument pruža pregled arhitekture sistema pumpd, uključujući dekompoziciju kroz dijagram paketa, dijagram komponenata i dijagram razmještaja. Prikazan je i konceptualni model baze podataka (perzistentni sloj), tabela prava pristupa za sve korisničke uloge, te opis kontrole toka i graničnih stanja sistema.

2 Arhitektura sistema

2.1 Kratak pregled arhitekture i funkcionalnosti podsistema



Sistem je zasnovan na dvoslojnoj klijent-server arhitekturi sa jasnom separacijom između mobilnog klijenta i serverske strane. Ova arhitektura dijeli sistem na tri logički odvojena segmenta:

- Sloj prezentacije (Presentation Layer): Mobilna aplikacija razvijena u React Native tehnologiji, dostupna na Android i iOS platformama. Prati MVVM arhitekturni obrazac, gdje je View zadužen za prikaz, ViewModel upravlja stanjem i logikom prikaza, a Model reprezentuje domenske podatke. Korisniku pruža sve ekrane i forme kroz jedinstven, prilagodljiv interfejs.
- Sloj aplikativne logike (Business Logic Layer): Pozadinski server implementiran kao Spring Boot aplikacija. Sadrži sve poslovne regule i logiku sistema. Izlaže REST API putem kojeg mobilna aplikacija komunicira s njim. Upravlja autentifikacijom, autorizacijom i validacijom podataka.
- Sloj podataka (Data Layer): Relaciona baza podataka (MySQL) za trajno čuvanje svih podataka sistema. Pristup bazi je isključivo putem aplikativnog servera, ne direktno s klijenta.

2.1.1 Tehnologije i razlog odabira

Za razvoj ovog sistema predložen je sljedeći tehnološki stek, odabran prema prirodi projekta — mobilna multiplatformska aplikacija s backendom:

Komponenta	Tehnologija	Uloga u sistemu
Mobilna aplikacija	React Native	Okvir koji omogućava pisanje jednog kôda koji se kompajlira u native Android (.apk) i iOS (.ipa) aplikacije. Koristi JavaScript/TypeScript jezik. Izabran jer sistem mora biti dostupan na obje platforme bez dupliranja posla.
Upravljanje stanjem	Redux Toolkit	Biblioteka za centralizovano upravljanje podacima unutar mobilne aplikacije — pamti ko je prijavljen, koji su podaci učitani i sl.
Navigacija	React Navigation	Biblioteka za kretanje između ekrana unutar aplikacije (prelaz s liste na detalje, otvaranje modala i sl.).
HTTP komunikacija	Axios	Biblioteka za slanje zahtjeva mobilne aplikacije prema REST API serveru. Upravlja HTTPS vezama i JWT tokenima.
Backend server	Spring Boot (Java)	Okvir koji olakšava pravljenje Java servera. Automatski konfigurira sve komponente i uključuje ugrađeni Tomcat web server, pa nema potrebe za posebnim serverskim softverom.
Sigurnost	Spring Security + JWT	Modul koji štiti API endpointe: provjerava ko je korisnik (autentifikacija) i šta smije raditi (autorizacija) na osnovu svoje uloge.
Pristup bazi	Spring Data JPA / Hibernate	Alat koji prevodi Java objekte u SQL upite i obratno — programeri pišu Java kôd umjesto SQL upita. Hibernate je konkretna implementacija, a JPA je standard koji se koristi.
Baza podataka	MySQL	Relaciona baza podataka otvorenog kôda, pouzdana i prikladna za produkcijsko okruženje. Čuva sve podatke sistema trajno.
Build alat (backend)	Maven	Alat koji upravljanje zavisnostima (bibliotekama) i izgradnjom Java projekta — preuzima sve potrebne biblioteke i pakuje server u izvršni .jar fajl.

2.1.2 Klijentski podsistem (mobilna aplikacija)

Mobilna aplikacija razvijena je u React Native-u, što znači da jedan kôd proizvodi aplikaciju za obje platforme. Aplikacija prati MVVM arhitekturni obrazac koji je dominantan u razvoju mobilnih aplikacija:

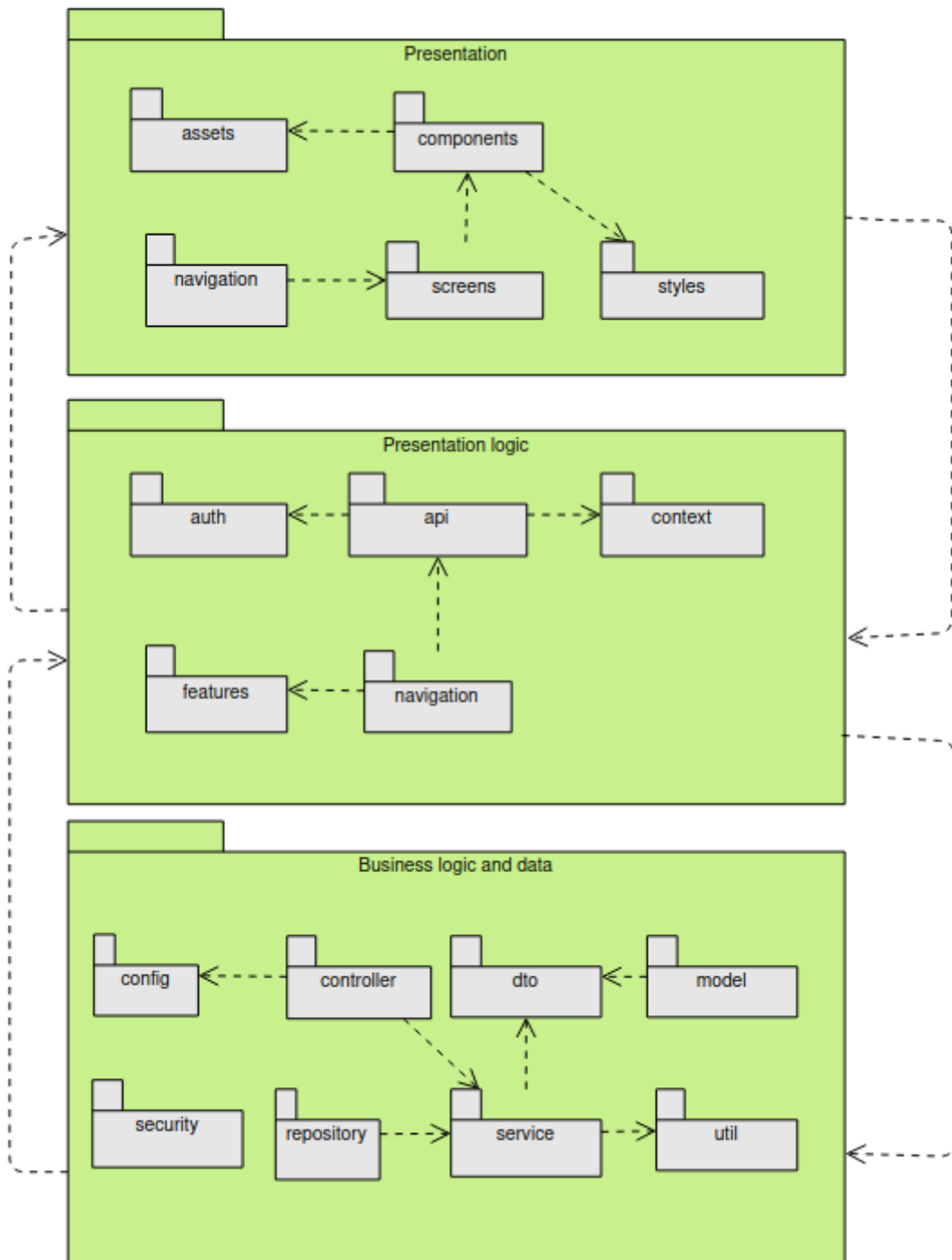
- View (Pogled): Ekran i UI komponente koje korisnik vidi i s kojima interaguje — liste, forme, kartice, dugmad. Svaki ekran je zaseban React Native komponent.
- ViewModel: Logika vezana za prikaz podataka — šta prikazati, kako formatirati, kada pozvati API. Implementirano kroz Redux slojeve i prilagođene React hooks-ove.
- Model: Strukture podataka koje reprezentuju entitete sistema (Član, Trener, Plan treninga, itd.) i servisi koji šalju zahtjeve prema REST API-ju.

2.1.3 Backend podsistem

Backend podsistem implementiran je kao Spring Boot aplikacija i prati četveroslojnu arhitekturu:

- Controller sloj: Ulazna tačka za sve HTTP zahtjeve s mobilne aplikacije. Svaki REST endpoint je definisan u odgovarajućem kontroleru. Vršiti osnovu validaciju i prosleđuje zahtjev servisnom sloju.
- Service sloj: Središnji sloj s poslovnom logikom. Koordinira između repozitorijuma, primjenjuje poslovne regule i priprema podatke za odgovor klijentu.
- Repository sloj: Koristi Spring Data JPA za pristup bazi podataka. Pruža metode za CRUD operacije bez pisanja direktnog SQL-a.
- Model sloj: Java klase koje mapiraju tabele baze podataka na objekte. Koriste Hibernate/JPA anotacije.

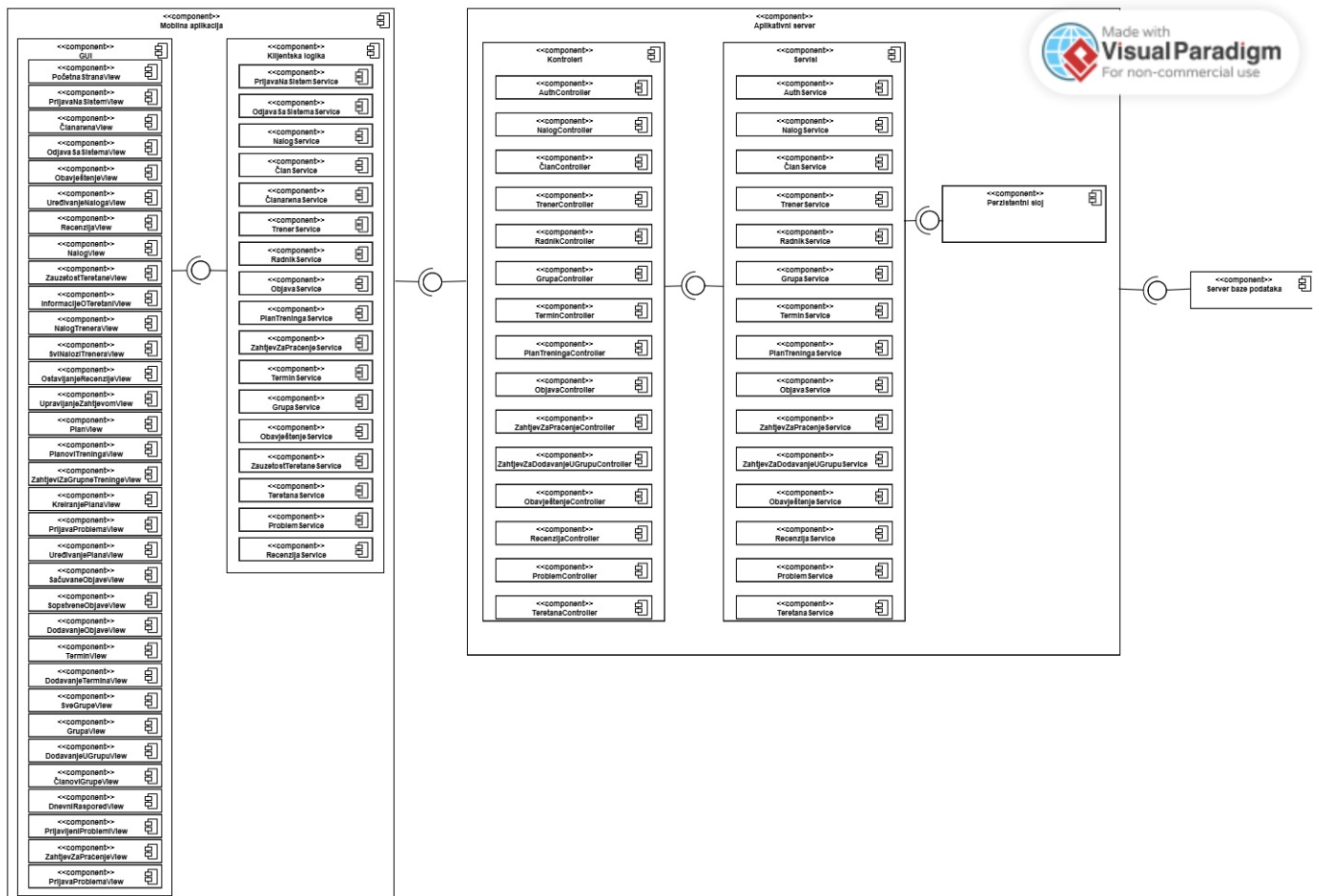
2.2 Dekompozicija sistema



Arhitektura informacionog sistema prikazana je dijagramom paketa i organizovana prema troslojnom arhitektonskom pristupu. Sistem je podijeljen na prezentacioni sloj, sloj prezentacione logike te sloj poslovne logike i podataka. Svaki od slojeva sadrži odgovarajuće pakete i komponente koje imaju jasno definisane odgovornosti i međusobne zavisnosti.

2.3 HW/SW mapiranje

2.3.1 Dijagram komponentata



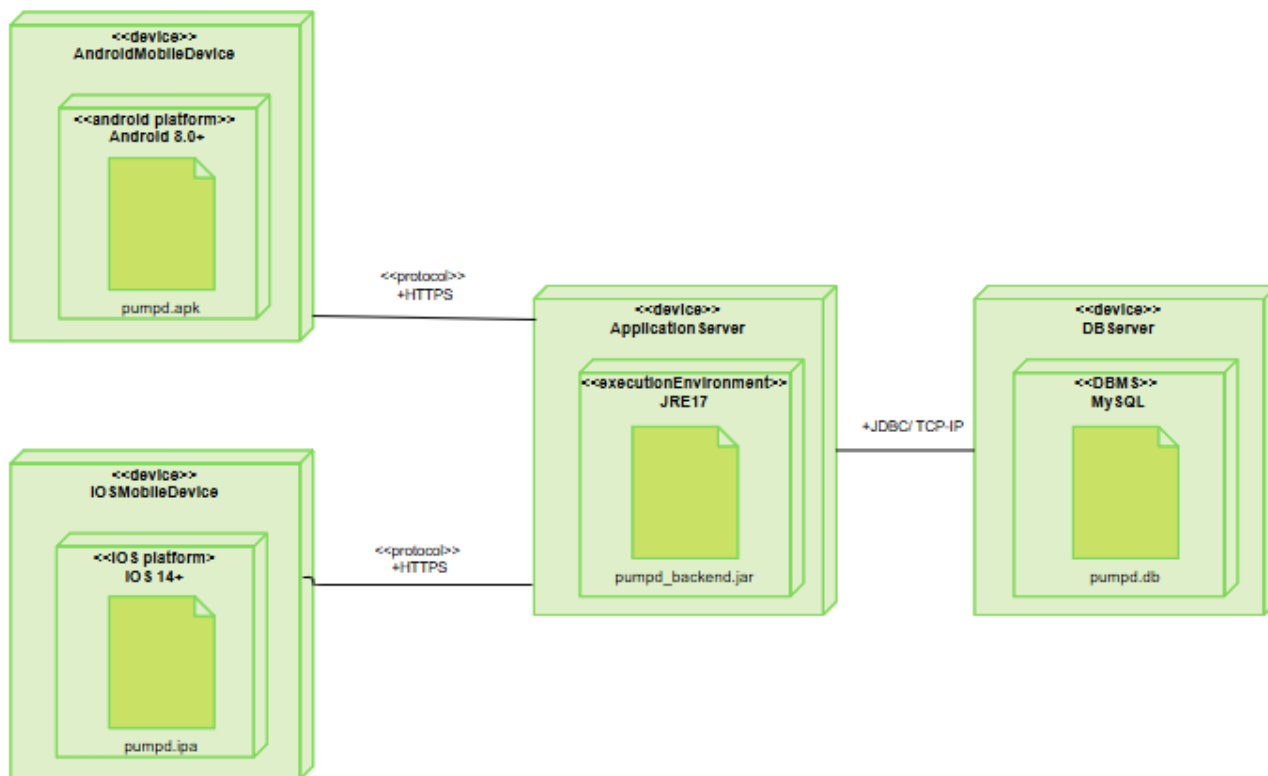
Dijagram komponentata modeluje strukturu sistema pumpd i zavisnosti između komponentata. Sistem se zasniva na klijent-server modelu s jednim tipom klijenta — mobilnom aplikacijom.

Mobilna aplikacija je jedini klijentski podsistem. Sastoji se od dva podsistema — GUI sloja koji prikazuje podatke korisniku putem ekrana, i klijentske logike koja komunicira s backend serverom putem REST API-ja.

Aplikativni server je centralna komponenta sistema. Sadrži tri podsistema — kontrolere kao ulaznu tačku za zahtjeve, servise koji implementiraju poslovnu logiku, i perzistentni sloj koji upravlja pristupom podacima kroz repository komponente.

Server baze podataka trajno čuva sve podatke sistema kao što su podaci o korisnicima, planovima treninga, terminima, recenzijama, objavama i slično.

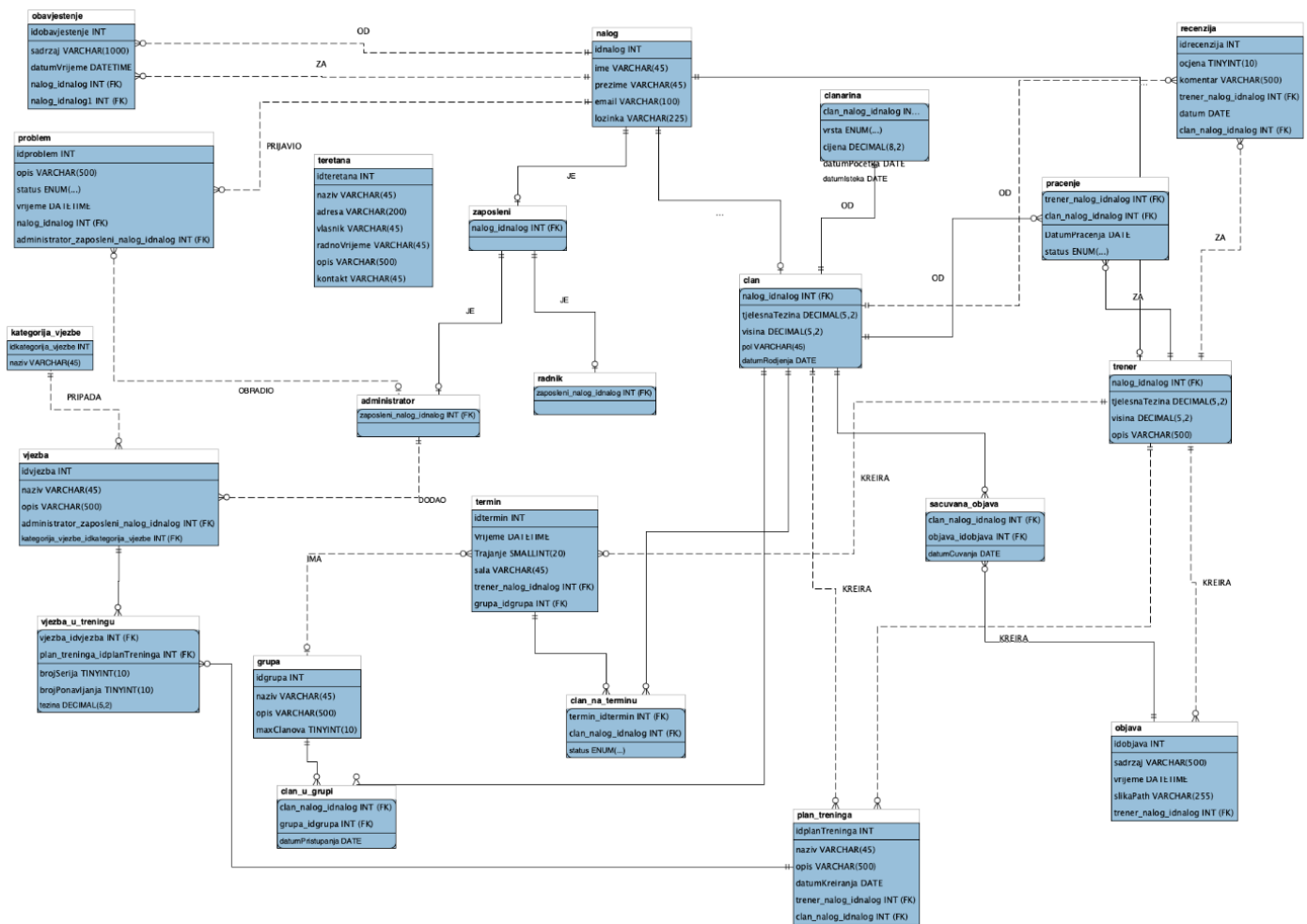
2.3.2 Dijagram razmještaja



Dijagram razmještaja prikazuje fizičku arhitekturu i topologiju sistema “pumpd”. Sistem je podijeljen na sljedeće hardverske i softverske komponente:

- **Klijentski sloj (Mobilni uređaji):** Android i iOS pametni telefoni/tableti. Na svakom uređaju izvršava se odgovarajući instalacioni paket aplikacije (pumpd.apk za Android, pumpd.ipa za iOS). Aplikacija komunicira s aplikativnim serverom putem HTTPS protokola što omogućava sigurnu komunikaciju.
- **Aplikativni server:** Centralni dio sistema koji obrađuje zahtjeve klijentskih aplikacija i izvršava poslovnu logiku. Java okruženje (JRE 17) u kojem se izvršava pumpd_backend.jar.
- **Server baze podataka:** Sloj za skladištenje i upravljanje podataka. Sistem za upravljanje bazom podataka je MySQL. Komunikacija s aplikativnim serverom odvija se putem JDBC drajvera (TCP/IP), što osigurava siguran i efikasan prenos podataka.

2.4 Perzistentni sloj



Prethodna slika predstavlja konceptualni model baze podataka na visokom nivou apstrakcije. Na osnovu njega implementiran je perzistentni sloj unutar aplikacije korišćenjem Hibernate/JPA alata za ORM mapiranje. Kao sistem za upravljanje relacionim bazama podataka odabran je MySQL.

2.5 Kontrola prava pristupa

Pristup funkcionalnostima sistema pumpd zasnovan je na ulogama korisnika (Role-Based Access Control - RBAC). Svaka uloga ima definisan skup privilegija nad funkcionalnostima sistema, čime se osigurava integritet podataka i sigurnost poslovanja.

Korisničke uloge u sistemu su hijerarhijski organizovane:

- Neprijavljeni korisnik ima pristup isključivo javnim informacijama sistema.
- Član i Trener su specijalizovane uloge prijavljenog korisnika namijenjene operativnom radu unutar teretane.
- Radnik i Administrator nasljeđuju klasu Zaposleni i imaju pristup funkcijama upravljanja sistemom.
- Administrator upravlja konfiguracijom sistema.

Učesnici Klase	Operacija	Neprijavljeni korisnik	Član	Trener	Radnik	Administrator
Član	dodajČlana()				■ ■ ■	■ ■ ■
	pregledajČlana(id)				■ ■ ■	■ ■ ■
	pregledajSveČlanove(list taČlanova)				■ ■ ■	■ ■ ■
	ukloniČlana(id)				■ ■ ■	■ ■ ■
	urediČlanarinu(id)				■ ■ ■	■ ■ ■
Trener	dodajTrenera()					■ ■ ■
	pregledajTrenera(id)		■ ■ ■			■ ■ ■
	pregledajSveTrenere(list aTrenera)		■ ■ ■			■ ■ ■
	ukloniTrenera(id)					■ ■ ■
Radnik	dodajRadnika()					■ ■ ■
	pregledajRadnika(id)					■ ■ ■
	pregledajSveRadnike(list taRadnika)					■ ■ ■
	ukloniRadnika(id)					■ ■ ■
Recenzija	ostaviRecenziju(ocjena)		■ ■ ■			
	pregledRecenzije()			■ ■ ■		
Grupa	dodajČlanaUGrupu()			■ ■ ■		
	pregledajGrupu()			■ ■ ■		
Objava	pregledajSopstveneObjave()			■ ■ ■		
	obrišiSopstvenuObjavu()			■ ■ ■		
	dodajObjavu()			■ ■ ■		
PlanTreninga	pregledajPlanTreninga()		■ ■ ■	■ ■ ■		
	kreirajPlanTreninga()		■ ■ ■	■ ■ ■		
	pregledajPrethodnePlan ove()		■ ■ ■			
	urediPlan()		■ ■ ■	■ ■ ■		
	obrišiPlan()		■ ■ ■	■ ■ ■		
Obavještenje	pregledajObavještenja()		■ ■ ■	■ ■ ■	■ ■ ■	■ ■ ■
ZahtjevZaPra ćenje	pregledajZahtjev()			■ ■ ■		
	upravljaZahtjevom()			■ ■ ■		

	pošaljiZahtjev()		■—■			
Problem	pregledajPrijavljeneProbleme()					■—■
	prijaviProblem()	■—■	■—■	■—■	■—■	
Termin	pregledajTermine()		■—■	■—■	■—■	■—■
	dodajTermin()			■—■		
	ukloniTermin()			■—■		
	odjaviSeSaTermina()		■—■			
	prijaviSeNaTermin()		■—■			
	urediTermin()			■—■		
DnevniRaspored	pregledajRaspored()			■—■		
Nalog	prijaviSe()	■—■				
	pregledajNalog()		■—■	■—■	■—■	■—■
	urediNalog()		■—■	■—■	■—■	■—■
	pregledPodatakaOČlanarini()		■—■	■—■	■—■	■—■
	odjaviSe()		■—■	■—■	■—■	■—■
	dodajČlanaUGrupu()			■—■		
ZahtjevZaDodavanjeUGrupu	pregledajZahtjeve()			■—■		
	upravljajZahtjevom()			■—■		
Teretana	PregledajZauzetostTeretane()		■—■			
	pregledajInformacijeOTeretani()	■—■				
	urediPodatkeOTeretani()					■—■

2.6 Identifikacija konkurentnosti

S obzirom da će pumpd aplikaciju istovremeno koristiti više različitih tipova korisnika (članovi, treneri, radnici i administratori), identifikacija i upravljanje konkurentnim pristupom predstavlja ključni aspekt za stabilan rad sistema.

Sistem je dizajniran tako da omogući istovremeno korišćenje od strane velikog broja korisnika bez konflikta u pristupu podacima:

- Svaka korisnička uloga ima jasno definirane i odvojene funkcionalne tokove, čime se preklapanje zahtjeva svodi na minimum. Na primjer, dok član rezerviše termin, trener može istovremeno dodavati novi plan treninga, a radnik ažurira status članarine — sve to bez međusobnog ometanja.
- Baza podataka (MySQL) koristi transakcioni pristup podacima i mehanizme zaključavanja redova (row-level locking) kako bi se očuvala konzistentnost pri istovremenim izmjenama.
- Spring Boot backend podržava višenitni rad kroz Tomcat thread pool, što znači da se zahtjevi više korisnika obrađuju paralelno, svaki u zasebnoj niti.
- Connection pooling (HikariCP, ugrađen u Spring Boot) efikasno upravlja vezama prema bazi podataka, čime se optimizira korišćenje resursa pri velikom broju istovremenih zahtjeva.

2.7 Kontrola toka

U sistemu pumpd, kontrola toka aplikacije centralizovana je na backend serveru i zasnovana je na HTTP zahtjev-odgovor modelu. Mobilna aplikacija inicira sve akcije, a server odgovara na zahtjeve.

Tipičan tok zahtjeva odvija se na sljedeći način:

- Korisnik interaguje s mobilnom aplikacijom (pritisak na dugme, unos podataka u formu, prevlačenje lista).
- Aplikacija formira odgovarajući HTTP zahtjev (GET, POST, PUT, DELETE) i šalje ga prema REST API-ju aplikativnog servera putem HTTPS protokola, uz priložen JWT token u Authorization zaglavlju.
- Spring Security filter na serveru presreće zahtjev, validira JWT token i provjerava da li korisnik ima dozvolu za traženu akciju.
- DispatcherServlet usmjerava zahtjev na odgovarajući Controller koji obrađuje zahtjev, poziva relevantni Service, koji zatim komunicira s Repository slojem za čitanje ili pisanje podataka u bazu.
- Server formira odgovor (JSON format) i vraća ga mobilnoj aplikaciji putem HTTPS protokola.
- Mobilna aplikacija obrađuje odgovor — ažurira Redux Store (centralno stanje aplikacije) i osvježava prikaz na ekranu korisnika.

Komunikacija je asinhrona — mobilna aplikacija ne blokira UI dok čeka odgovor servera. Umjesto toga, prikazuje indikator učitavanja (spinner) i obrađuje odgovor kada stigne, što osigurava gladak korisnički doživljaj.

2.8 Granična stanja sistema

2.8.1 Inicijalizacija

Sistem pumpd sastoji se od više međusobno povezanih komponenti koje se mogu nezavisno pokrenuti. Za potpunu funkcionalnost aplikacije neophodna je aktivnost svih komponenti.

Inicijalizacija sistema odvija se u sljedećim fazama:

- Priprema serverskog okruženja: Instalacija JRE 17, MySQL i konfiguracija mrežnog okruženja.
- Konfiguracija baze podataka: Kreiranje baze podataka (pumpd_db), primjena šeme migracija (Flyway/Liquibase), unos inicijalnih podataka (npr. podaci o teretani, kreiranje admin naloga).
- Pokretanje backend servera: Pokretanje pumpd_backend.jar — Spring Boot automatski inicijalizuje sve komponente, uspostavlja vezu s bazom podataka i pokreće ugrađeni Tomcat server.
- Distribucija mobilne aplikacije: Objava pumpd.apk na Google Play Store i pumpd.ipa na Apple App Store, odakle korisnici preuzimaju aplikaciju na svoje uređaje.
- Inicijalno pokretanje aplikacije na uređaju: Aplikacija provjerava dostupnost internet veze i prikazuje ekran za prijavu ili registraciju.

2.8.2 Terminiranje

S obzirom na to da je pumpd sistem koji treba biti dostupan 24/7, terminiranje se ne predviđa kao rutinska operacija, već samo u slučaju planiranog održavanja ili neočekivanih otkaza.

Planirana terminiranja (održavanje servera) uključuju sljedeće korake:

- Slanje obavještenja korisnicima o planiranom prekidu rada (putem push notifikacija ili obavještenja unutar aplikacije).
- Spring Boot automatski zatvara sve aktivne konekcije prema bazi podataka i oslobađa zauzete resurse.
- Sva zauzeta memorija i otvoreni mrežni portovi bivaju oslobođeni.
- Nakon završetka održavanja, sistem se ponovo pokreće bez ikakvih problema.

2.8.3 Otkazi

Sistem je projektovan tako da bude otporan na otkaze i da u slučaju grešaka reaguje na pouzdan i predvidljiv način:

- Gubitak internet veze na mobilnom uređaju: Aplikacija detektuje prekid veze i prikazuje korisniku odgovarajuću poruku. Određeni sadržaj (prethodno učitani podaci) može biti vidljiv iz cache memorije.
- Nedostupnost backend servera: Mobilna aplikacija prikazuje poruku o grešci i nudi opciju ponovnog pokušaja. Spring Boot Actuator može biti konfigurisan za monitoring i automatski restart u slučaju pada.
- Greška u bazi podataka: MySQL transakcioni mehanizam osigurava da se nepotpune transakcije automatski poništavaju (rollback), čuvajući konzistentnost podataka.
- Neispravi korisnički unos: Validacija se vrši na dva nivoa — na mobilnoj aplikaciji (korisnički interfejs) i na backend serveru (servisni sloj), što osigurava da neispravni podaci nikad ne dostignu bazu podataka.
- Istekli JWT token: Aplikacija automatski pokušava osvježiti token putem refresh mehanizma. Ukoliko to nije moguće, korisnik se preusmjerava na ekran za prijavu.

Cilj je da se svi prekidi u radu svedu na minimum kako bi se osigurao pouzdan rad aplikacije i što bolje korisničko iskustvo za sve korisnike sistema pumpd.